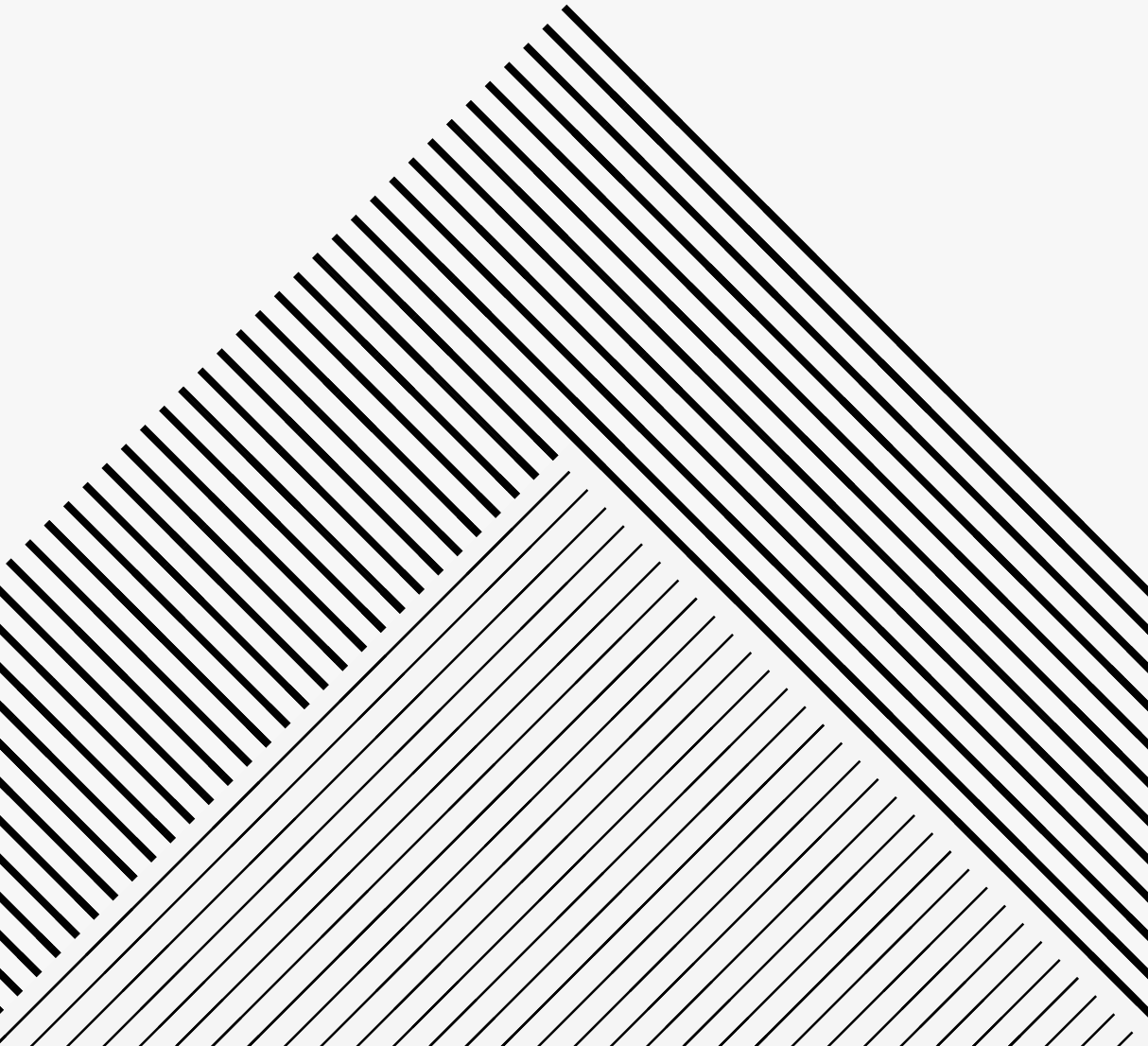


1. Kolokvij

Vrste uporabniških vmesnikov:

Konzolno usmerjen uporabniški vmesnik

Akcija → objekt

- navadna konzola
- POSTOPKOVNO PROGRAMIRANJE
  programer določi kdaj bo prišlo do dogajanja (kdaj upiše podatke, dobi rez.)

Grafično usmerjen uporabniški vmesnik

Objekt → akcija

- dogodkovni interaktivni vmesnik (XM)
- DOGOTKOVNO USMERJENO PROGRAMIRANJE
  programer implementira UV to pomeni dobči odzive na upor. akcije (nezaporedno)



Konzolno Usmerjeni UV

Navadni konzolni vmesniki

- brez neposredne interakcije
- linearni
- vhodni parametri

Konzolni interaktivni vmesniki

- neposredna interakcija
- nelinearni
- veliko neizvajanja (vmesnik čaka uporabnikov vnos)

Dogotkovno Interaktivni UV

Preprost Dog.-Int. UV

- neposredna interakcija
- nelinearni
- veliko neizvajanja
- uporabnik lahko kadarkoli dela karkoli
- dejanjem rečemo DOGODKI
- aplikacija čaka dogodke v zanki

Z grafičnim UV

- Potrebna registracija rokovalnika dogodkov
- Dogodke čakajo rokovalniki, sprožijo ozivne funkcije

Dogodki

- Dogodki so objekti, kreirani kot odziv
- Več dogodkov hkrati lahko vodi do težav
- Dogodki se hranijo v vrsti
- Premik miške je 1 sam dogodek

Vrste

Preprosti vhodni dogodki	Prevedeni vhodni dogodki
- premik miške - LMB - tipkovnica	- (dvojni) klik miške - vstop/iztop miške iz komponente - gained/lost fokus - odtipkan znak (ko veljavno)

- Včasih se preprosti dogodki pretvorijo v prevedene

ZAGOTAVLJANJE ZANKE

napredna orodja - interno

preprosta orodja - aplikacija sama

Okenski sistem X (X11)

Zahteve:

- povezati množico grafičnih delovnih postaj
- poljuben sistem
- oddaljen sistem

Alternative

- Wayland (Linux)
- Desktop window manager (Windows)
- Quartz Compositor (Mac OS)

Lastnosti – Prvo strojno in pozicijsko neodvisno okolje

- neodvisno od OS
- grafično okolje veliko sistemov (Solaris, AIX, HPUX)
- sistem okna
- koncept odjemalec/strežnik (lahko ločena)
- star, redka uporaba (toda delujr brez strojnega pospeševanja)

Kako deluje:

- Prikazovalnik / display (abstrakcija) - predstavlja V/O uporabnika
- Strežnik nadzoruje prikazovalnik: (not local)

  - dostop za uporabnike
  - pošilja sporočila
  - sledi zahtevam
  - posodablja okna
  - dvodimenzionalno risanje

- Odjemalci imajo dostop do prikazovalnika

  - izvajajo sočasno (več oken / več računalnikov / neodvisnost aplikacij)
  - uporabljajo mrežni protokol (preko vmesnika Xlib)
  - med sabo komunicirajo preko strežnika

Orodja za Načrtovanje UV

- Knjižnice razredov, ki dajo predloge za graf. el.
- objektom teh razredov lahko spreminjamo lastnosti
- vsebovalniki lahko vsebujejo več podob

- GTK+, Qt, wxWidgets
- MFC, Cocoa, Motif
- Swing, Awt, SWT, JavaFX

Wayland

Je protokol, ki določa komunikacijo med strežnikom (zaslon) in odjemalci (tudi C knjižnica)

Uporablja dva nivoja:

- nizkonivojski: medprocesna komunikacija
- visokonivojski: izmenjava informacij med odjemalci in kompozitorjem za okenski sistem (generiran avtomatsko)

Osnovni elementi:

wl_display, wl_registery, wl_compositor, wl_surface, wl_buffer, wl_output, wl_pointer, wl_keyboard, ...

Značilnosti:

- prikazovalnik in kompozitor sta združena
- upodabljanje se prepusti odjemalcu (pisave, podobe, ...)
- loči I/O vsakega okna, kar zagotavlja zanesnost, integriteto
- Ne zagotavlja medprocesne komunikacije in omrežne transparentnosti

MVC - Model/View/Controller

Model

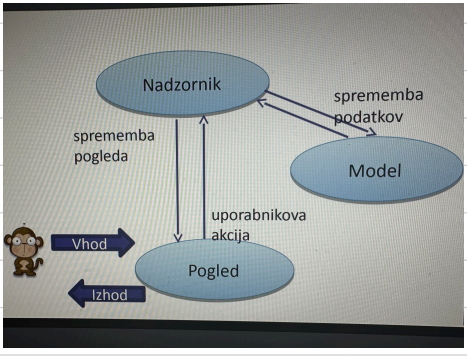
- Podatki povezani z aplikacijo
- Metode nad podatki/stanji
- Ni povezan z vmesnikom

ZAKAJ?

- Omogoča gradnjo generičnih podob
- Podatki so ločeni od prikaza
- Lažje vzdrževanje
- Lažji reuse komponent
- Kompleksnost se porazdeli
- Povečana fleksibilnost

Pogled

- Prikaže vsebino modela
- Omogoča uporabniku upravljanje
- Prilagaja se spremembam modela



Nadzornik

- Posrednik med Modelom in Pogledom
- Prevede akcije v operacije na modelu
- Posodobi model

Java Grafični Uporabniški Vmesnik

AWT - Abstract Window Toolkit

- Default
- Platformno odvisno (različne enakovredne komponente na različnih platformah)
- Abstraktno (prilagodljivo in prenosljivo ogrodje razredov)
- Gradnike izriše operacijski sistem

Swing

- Upgrade from AWT
- Namenjeno razvoju namiznih aplikacij
- Razredi izpeljani iz AWT
- Ni specifičen za platforme
- OS zagotovi risalno površino/pomnilnik
- Java VM narišе lahke gradnike
- Težki gradniki še vedno od OS

JavaFX

- Swing alternative (swing prezapleten za razvoj)
- Prvotno namenjen spletnim aplikacijam
- JavaFX omogoča dodajanje Swing komponent (in obratno)
- JavaFX spremlja izgled samo s CSS

Prednosti in Slabosti JavaFX

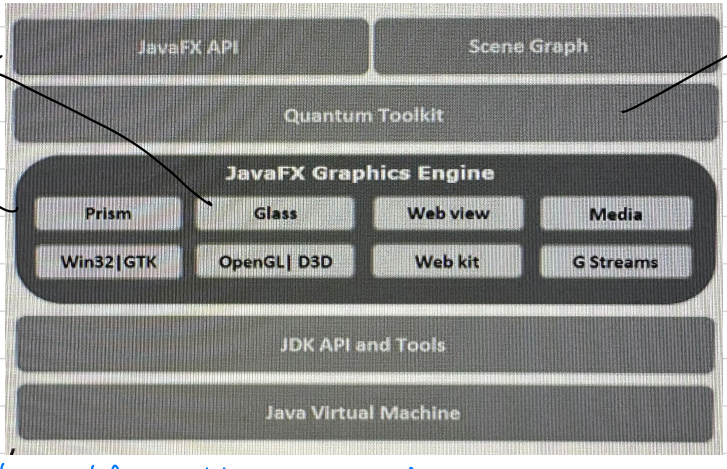
Prednosti:

- Relativno moderen (2008)
- Podpora FXML
- Uporaba Swing
- Komponente za delo s HTML
- JavaScript interpreter
- Medijske komponente

Slabosti:

- Razvoj ločen od Jave
- Posebna namestitev
- Težko za postaviti aplikacijo

Arhitektura JavaFX



Okenski sistem

Grafični cevovod

Abstrakcijska plast

Komponenta za vključevanje HTML5 in Media

JavaFX uporablja metaforo gledališča za grafične aplikacije:

oder (javafx.stage.Stage) – vrsni vsebovalnik (okno)

scena (javafx.scene.Scene) – več komponent v sceni, aplikacija lahko več scen (1 na odru)

Gradnja Uporabniškega Vmesnika

- Pripravimo scenski graf
- Konstruiramo sceno s korenskim vozliščem → običajno razvrščevalnik, ki razvršča svoje otroke
- Pripraviti oder s konstruirano sceno

Uporabnost

- Uporabniški vmesnik predstavlja prvi stik
- UV vpliva na sprejetost programske opreme (zavračanje neuporabnih)
- Uporabnik se lahko krivi za slab vmesnik

- Slabi vmesniki: frustracija, zbeganost, dolgčas, panika, slaba izkoriščenost sistema
- Uporabniški čas vedno bolj pomemben
- Napake so lahko katastrofalne

Načrtovanje

- Pomembna komunikacija uporabnik + programer
- Uporabnik ima vedno prav (konsistentne napake → slab vmesnik)
- Toda uporabniki ne vejo vedno kaj hočejo (WYKIWYL)

Definicija uporabnosti:
Obseg s katerim lahko določeni uporabniki uporabljajo izdelek za doseganje določenih ciljev z učinkovitostjo, na učinkovit način in zadovoljstvom in v določenem kontekstu uporabe.

Učinkovitost: natančnost in celostjo s katero lahko nek uporabnik doseže cilje

Učinkovit način: porabljeni resursi v povezavi z natančnostjo in celovitostjo

Zadovoljstvo: udobje in sprejemljivostjo delovnega sistema za uporabnika

Torej uporabnost je kako dobro uporabniki uporabljajo funkcionalnost sistema.
Treba je razmišljati o njej med razvojem.

Dimenzije uporabnosti

- Naučljivost: se je lahko za naučiti?
- Učinkovitost: ali se da hitro uporabljati ko naučeno?
- Varnost: ali so napake redke in popravljive?

(- Estetika: zadovoljstvo
- Ergonomika: utrujenost)

Te imajo različno pomembnost.

GLEDE NA UPORABNIKA:
- Novinec ceni naučljivost
- Ekspert ceni učinkovitost
- Popolni novinci in popolni eksperti NE OBSTAJAJO!

GLEDE NA NALOGO:
- Varnost izvedbe naloge
- Učinkovitost

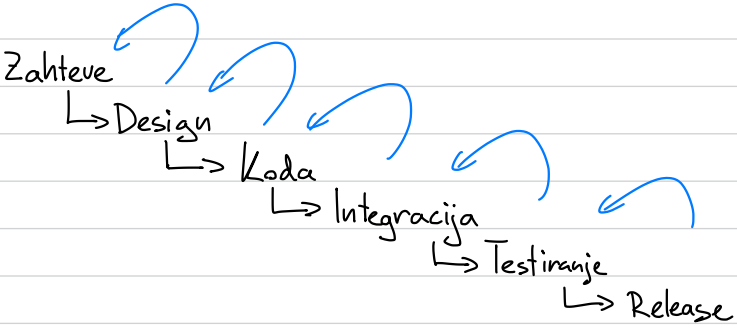
Naš glavni cilj je uporabnost. Upoštevati pa moramo tudi druge zahteve in delati kompromise

funkcionalnost, zmogljivost, zanesljivost, zaščita, cena, standardi, ...

Razvijanje Programske Opreme

Tradicionalno: Slapovni model

↳ Najprej premisli nato implementiraj

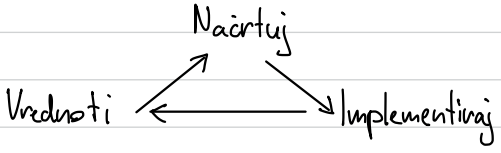


Možnost vračanja? ali pa od začetka

Problematika:

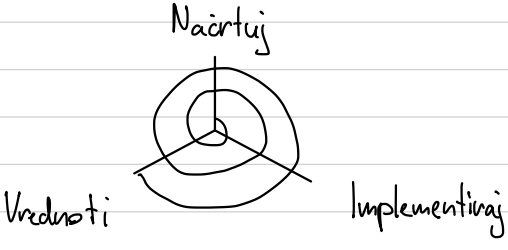
- načrtovanje vmesnikov je tvegano
- pojavijo se napake, ki se ne odkrijejo do testiranja
- kaj narediti ko se spremenijo zahteve (zavračanje kode drago in zamudno)

Iterativno načrtovanje



- Manj tvegano
- Več iteracij (ne vsaka vse)
- Napredek ocenjuje končni uporabniki

Spiralni model



- Inkrementalni model (add) ne iterativni (redo)
- več iteracij v enem procesu
- nadij = cena

Uporabniško usmerjeno načrtovanje UV

- Spiralni model
- Uporabniki morajo sodelovati v vsaki iteraciji
- Analiza uporabnikov
- Pri načrtovanju upoštevaj navodila
- Ocenjujemo s heuristiko ne trdimi pravili

IMPLEMENTIRANJE S PROTOTIPI!!

Sposobnosti človeka

Spomin:

- kratkotrajni senzorični spomin (STSS)
- dolgotrajni spomin (LTM)
- delovni spomin (WM)

Razlikujejo se v: tipu podatkov, velikosti, času pozabljanja

Procesiranje

- Zaznavanje ~100 ms
- Razumevanje ~ 70 ms
- Motorika ~ 70 ms

Velike razlike med populacijo, tudi do 10x.

Zaznavanje

- Primerjanje novih izkušenj z starimi (iz dolgotrajnega spomina)
- Kratkoročni spomin sprejema ogromno informacij (ne zavedamo)
- Ljudje potrebujejo vedno več dražljajev

Vidljivost

- Odvisna od mesta pozornosti (iskalni žaromet)
- Sekundarni dražljaji (npr. barva teksta) - nepovezano, vpliva na vidljivost
- $1/T_P$ fps (100ms) je dovolj za sprejem gibajoče slike → odziv manjši od T_P instant

- Zaznavanje spodaj navzgor
  A - dve vodoravni črti, ena horizontalna
  (dražljaji)

- Zaznavanje zgoraj navzdol
  A - prva črka abecede
  (dolgoročni spomin)

Reakcijski čas

perceptual cognitive motor

$$RT = T_p + T_c + T_M =$$

$$= 100 + 70 + 70 \cong 240\,ms$$

$$RT_c = c + d \cdot \log_2 (1/Pr(stimulus)) \qquad c, d = konst.$$

čas T za premik roke do tarče velikosti S na razdalji D je

$$T = RT + MT = a + b \log_2 (D/S + 1)$$

/

reakcijski čas premika roke

Tarče na robu - neskončna velikost ($T = a$)

Naloga vodenja: (linearna)

$$T = a + b \cdot D/S$$

Funkcija učenja:

$$T_n = T_1 \cdot n^{-\alpha} \qquad (\alpha \text{ med } 0{,}2 \text{ in } 0{,}6)$$

Interakcije

Ljudje potrebujejo čas za učenje ⟶ napaka - dialog - gumb OK

Ukazni jezik

- Pomembna sintaksa / nenaravni jezik
- Primeri (lupine, URL, Regex)

Meni

- Izbira iz menija
- Vnos v formo
- Navigacijska struktura

Direktna manipulacija

- Podaljšek naravnega sveta
- Fizične interakcije
- Takojšnji in vidni rezultati akcij
- Akcije povleči/spusti
- Čudne ikone
- Težko načrtovanje

Primeri:
- drsniki
- folder dragging
- window resize

Antropomorfni vmesniki

- Naravna komunikacija
- Jezik, geste, izraz, premik oči
- Pogovorni UV (chatbots, voice assistant)
  - splošno uporaben
  - brez privajanja
  - odgovor v kontekstu
  - prilagodljiv
  - malo vizualnih namigov
  - prompting is hard
  - primerno za enostavne naloge
  - dragi

Modeli

Skupek delov za en cilj

Model sistema
Model uporabnika
Model vmesnika

Normanovi namigi (povezava uporabnik - sistem)

- pomagljivost
- oznake
- omejitve
- naravne preslikave
- vidljivost
- povratne informacije

Pomagljivost/Dostopnost -> namig o moči uporabnika

Zaznava dostopnosti -> uporabnik zazna, kaj je možno

Dejanska dostopnost

Principi - koncepti za načrtovanje

Mandelovi principi:

1. Zagotovi nadzor uporabnika
2. Uporabnik naj ne rabi veliko spomina
3. Konsistentnost

Navodila - izhajajo iz principov

Standardi - karakteristike objektov v uporabi

NIELSONOVIH 10 PRINCIPOV

1. Prilagodi se realnemu svetu
   (splošne besede, ne omejuj imen, metafore)

2. Konsistentnost in standardi
   (podoben izgled → podobno obnašanje)

3. Pomoč in dokumentacija
   (uporabniki ne berejo priročnikov → pomoč jim mora biti na voljo)

4. Uporabnikov nadzor in svoboda
   (vedno opcija preklica/izhoda)

5. Vidljivost statusnega sistema
   (loading, statusna vrstica, vidljivost akcij/načinov, feedback)

6. Fleksibilnost in učinkovitost
   (bližnjice, nastavitve, zgodovina, text prediction)

7. Izogibanje napakam
   (meniji/forme, nemogoče akcije, provide info)

8. Raje prepoznaj, kot si zapomni
   (meniji, seznami, generični ukazi)

9. Javljanje napak, diagnoza, reševanje
(natančna obvestila, skrij tehnične podrobnosti, pomagaj)

10. Estetika in minimalizem
(manj je več)